PREVISÃO DE DEMANDA COM RANDOMFORESTREGRESSOR - TONELADAS TRANSPORTADAS EM FERROVIAS BRASILEIRAS

GUILHERME LIMA DE SOUSA – MBA DATA SCIENCE & ANALYTICS

Entre em contato comigo pelo Linkedin: www.linkedin.com/in/guilherme-lima-747355169

Introdução

Em 2019 tivemos um "boom" no transporte ferroviário de cargas, além dos inúmeros investimentos que foram anunciados para o setor. Com certeza um ano marcante para o modal, que em 2022 segundo o governo federal, (https://lnkd.in/di2ckPYm) participava de apenas 19% de toda a operação de transporte de cargas. Sendo estratégico para o escoamento de grandes quantidades, as ferrovias se mostram mais seguras e mais rápidas quando comparados ao modal rodoviário. Trazendo maior agilidade nas operações logísticas de transporte de cargas.

Objetivo

Este estudo almeja prever a quantidade de toneladas transportadas nas ferrovias Brasileiras em 2023, utilizando RandomForestRegressor da biblioteca scikit-learn no Python. Com dados de 2006 a 2022, coletados no site oficial da ANTT – AGÊNCIA NACIONAL DE TRANSPORTES TERRESTRES (https://dados.antt.gov.br/).

Material e Métodos

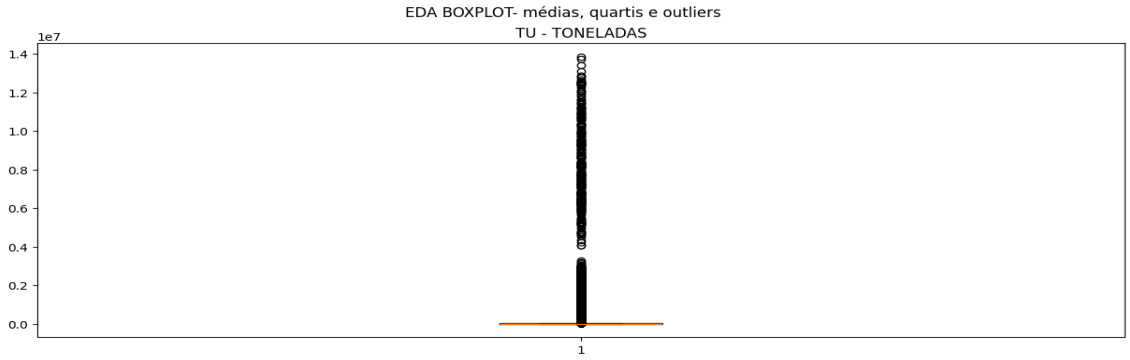
Após a obtenção dos dados, foi realizado o tratamento (ETL) e unificação das bases.

Abaixo é observado as principais características do conjunto de dados:

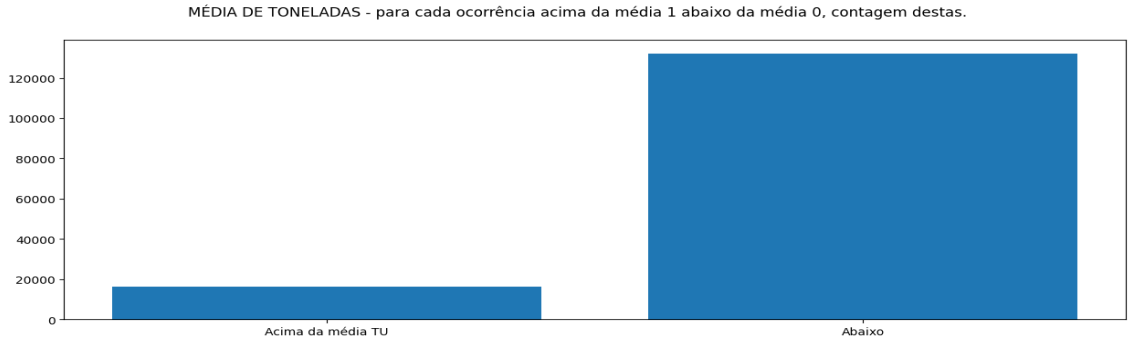
```
base_original.describe()
```
✓ 0.5s

	Mes_Ano	TU	TKU	MÊS	ANO
count	148629	148629.00	148629.00	148629.00	148629.00
mean	2013-12-14 09:31:35.153704960	54622.61	19498042.43	6.47	2013.50
min	2006-01-01 00:00:00	0.00	-2147483648.00	1.00	2006.00
25%	2009-05-01 00:00:00	981.00	392896.00	3.00	2009.00
50%	2013-06-01 00:00:00	4490.00	2047935.00	6.00	2013.00
75%	2018-08-01 00:00:00	16912.00	8870595.00	9.00	2018.00
max	2023-04-01 00:00:00	13858719.00	1920541926.00	12.00	2023.00
std	NaN	401341.54	136936386.47	3.46	5.17

Depois é realizado a análise exploratória dos dados (EDA) com foco na coluna target, por meio de BOXPLOT:

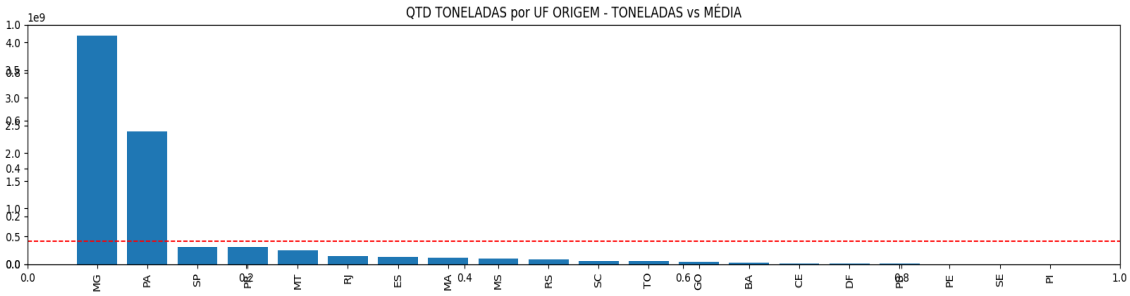


É verificado também quantos registros (contagem) possuem valor acima ou abaixo da média:

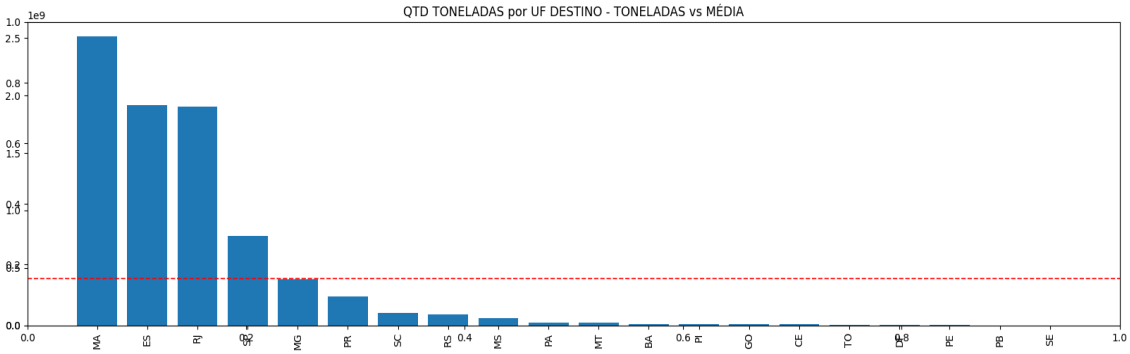


O que podemos observar até aqui é que nosso conjunto de dados está bastante diverso, isso é positivo, porém com muitos registros abaixo da média de toneladas e com outliers grandes.

Podemos observar a representatividade no volume de toneladas com foco na **UF de ORIGEM**:



Podemos observar a representatividade no volume de toneladas com foco na **UF de DESTINO**:



Os estados **MG, PA, SP** são os que mais enviam produtos pelas ferrovias e os estados **MA, ES, RJ** são os que mais recebem .... Com estes primeiros diagnósticos é possível compreender as características atuais do nosso conjunto de dados.

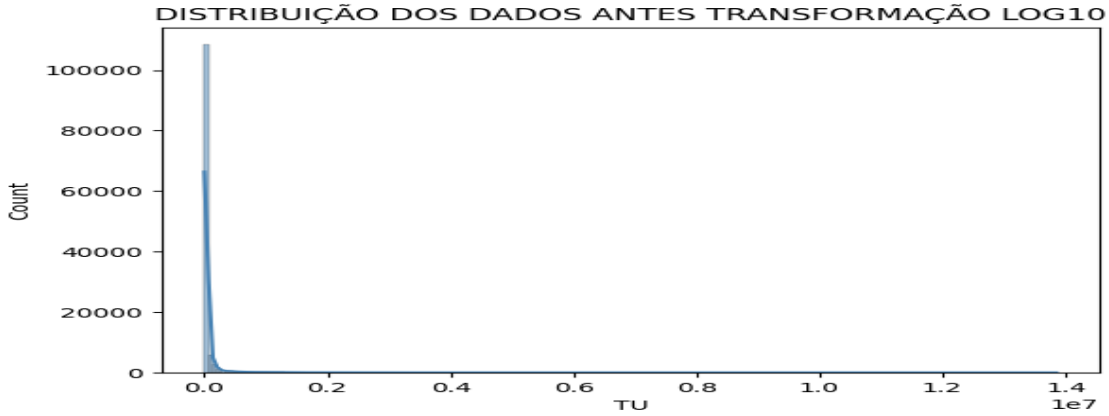
Foram realizadas transformações nas colunas de texto, aplicando códigos correspondentes a cada uma, pois o modelo só aceita colunas numéricas. Sendo assim, após as transformações, temos as colunas necessárias para compor o treinamento do modelo:

-MÊS -ANO -Codigo_Ferrovia -Codigo_Mercadoria_ANTT -Codigo_Estacao_Origem

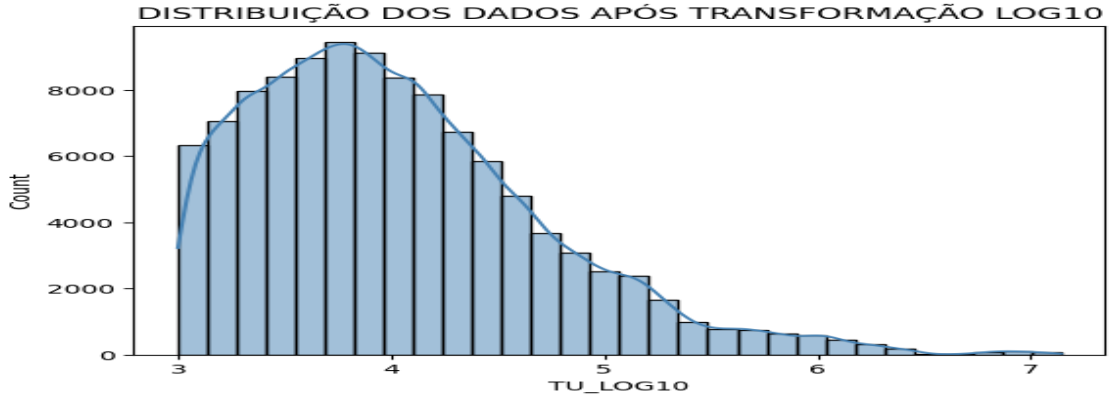
-Codigo_UF_Origem -Codigo_Estacao_Destino -Codigo_UF_Destino -TU

Como o nosso conjunto de dados é bastante complexo a sua distribuição não está muito próxima da normal e para ajustar essa distribuição, será feita uma transformação logarítmica de base 10 na coluna target (TU).

Observe a distribuição dos dados **antes da transformação logarítmica**:



Observe a distribuição dos dados **após a transformação logarítmica**:



Por fim, após a transformação logarítmica, temos as colunas necessárias para criação do modelo:

-MÊS -ANO -Codigo_Ferrovia -Codigo_Mercadoria_ANTT -Codigo_Estacao_Origem

-Codigo_UF_Origem -Codigo_Estacao_Destino -Codigo_UF_Destino -TU_LOG10

Treinando o Modelo

O modelo escolhido para predição dos valores foi o **RandomForestRegressor**, pois ele se adapta bem à conjuntos de dados complexos, como o nosso.

Na definição de parâmetros do modelo está sendo solicitado a criação de 10 árvores aleatórias, com profundidade máxima de 5 nós para cada árvore, e com reprodutibilidade ativado.

```
# CRIANDO MODELO (10 arvores, profundidade 5, replicabilidade seed 2)
modelo = RandomForestRegressor(n_estimators=10, max_depth=5, random_state=2)
```

Para assegurar ainda mais a performance do modelo, é definido o cross validation de 5 folds, com embaralhamento dos dados ativo, além da reprodutibilidade também ativo:

```
# VALIDAÇÃO CRUZADA
kf = KFold(n_splits=5, shuffle= True, random_state=2)
cross_validation = cross_val_score(modelo, x, y, cv=kf, scoring='r2')
```

Na validação cruzada o modelo é dividido em K-Folds (partes iguais), embaralhado e avaliado em cada etapa. Quando não é utilizado o método de validação cruzada, o modelo fica enviesado apenas aquela única divisão dos dados, o que pode limitar a sua performance. Então a validação cruzada permite que o conjunto de dados seja dividido em pequenas partes, com cada subconjunto distinto e treinando o modelo em cada um deles, permitindo observar melhor o desempenho total.

Após o treinamento do modelo foi obtido um resultado de 0,44 ou 44% de assertividade (R2). O que é considerado um resultado ruim, insuficiente para embasar decisões. Modelos com assertividade acima de 70% são mais assertivos para embasar decisões.

```
----------  RESULTADOS MODELO SIMPLES  ----------
O tipo do modelo carregado é: <class 'sklearn.ensemble._forest.RandomForestRegressor'>
Acuracia para cada fold (cross-validation): [0.44117468 0.43116143 0.44557165 0.44419751 0.42601058]
Acuracia média (cross-validation): 43.76231692179759
Desvio padrão (cross validation): 0.007689213316033354
R² no conjunto de teste: 0.44
MAE: 65142.186426917746
RMSE: 464205.69742769195
MAPE: 145.47%
MedAE: 5514.72
```

O que podemos fazer para melhorar o resultado do modelo?

Dentre as possibilidades que podemos empregar para melhorar o resultado deste modelo será utilizado o processo de **AUTO TUNNING** através do RandomizedSearchCV, que é uma classe dentro do scikit-learn para encontrar automaticamente os melhores hiperparâmetros para um modelo. Ele sorteia combinações aleatórias de hiperparâmetros dentro de intervalos que nós definimos e verifica qual combinação é melhor.

```
# Espaço de busca dos hiperparâmetros
param_dist = {
    'n_estimators': randint(100, 300),
    'max_depth': randint(1, 15),
    'min_samples_split': randint(1, 10),
    'min_samples_leaf': randint(1, 5),
    'max_features': ['auto', 'sqrt', 'log2'],
    'bootstrap': [True, False]
}

# Randomized Search
random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=50,                  # número de combinações aleatórias testadas
    cv=5,                       # validação cruzada
    scoring='r2',               # métrica
    verbose=2,
    random_state=42,
    n_jobs=-1
)
```

Após a realização do AUTO TUNNING os parâmetros indicados que melhoram a performance são atribuídos ao modelo:

```
########### Melhores hiperparametros:
{'bootstrap': False, 'max_depth': 20, 'max_features': 0.8, 'min_samples_leaf': 3, 'min_samples_split': 17, 'n_estimators': 152}
########### Melhor Score encontrado:
```

e com isso nosso resultado sobe para 0.86 ou 86% de assertividade (R2). Um resultado bem melhor do que o inicial:

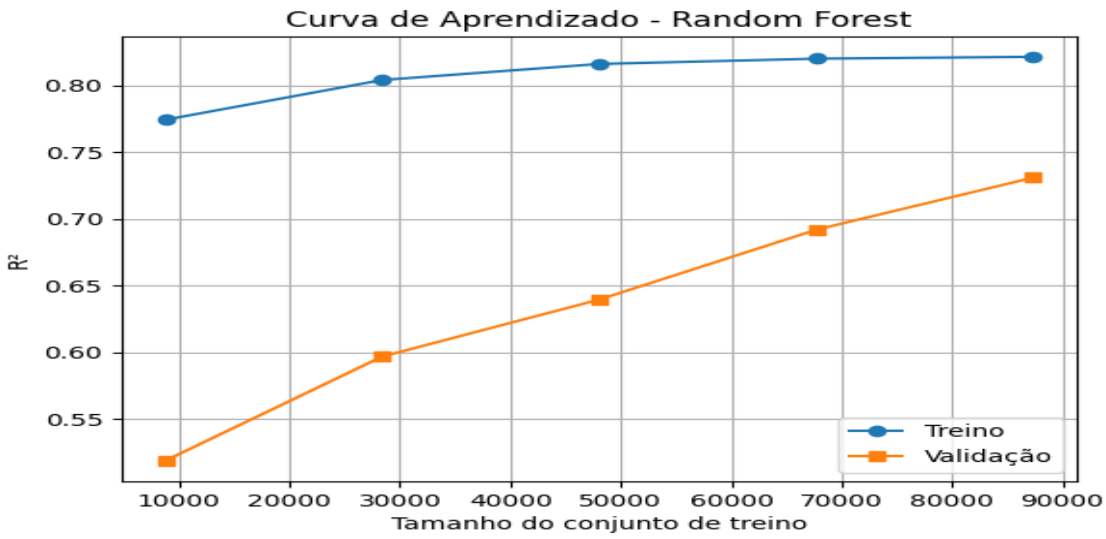
```
----------  RESULTADOS MODELO AUTO TUNNING  ----------
O tipo do modelo carregado é: <class 'sklearn.ensemble._forest.RandomForestRegressor'>
Desvio padrão (cross validation): 0.0003351117060657536
MAE médio (cross-validation): 0.0463
R²: 0.87
MAE: 16041.073483454851
MSE (Mean squared error): 5781100696.830794
RMSE (Root Mean Squared Error): 76033.54981079599
MAPE: 53.17%
MedAE: 2334.37
```

Posteriormente o modelo é salvo no diretório para podermos utilizar no estudo de caso.

Avaliando o Modelo

Para um modelo ser considerado realmente bom, outras métricas além da assertividade (R2) precisam ser avaliadas. Abaixo temos as avaliações destas outras métricas:

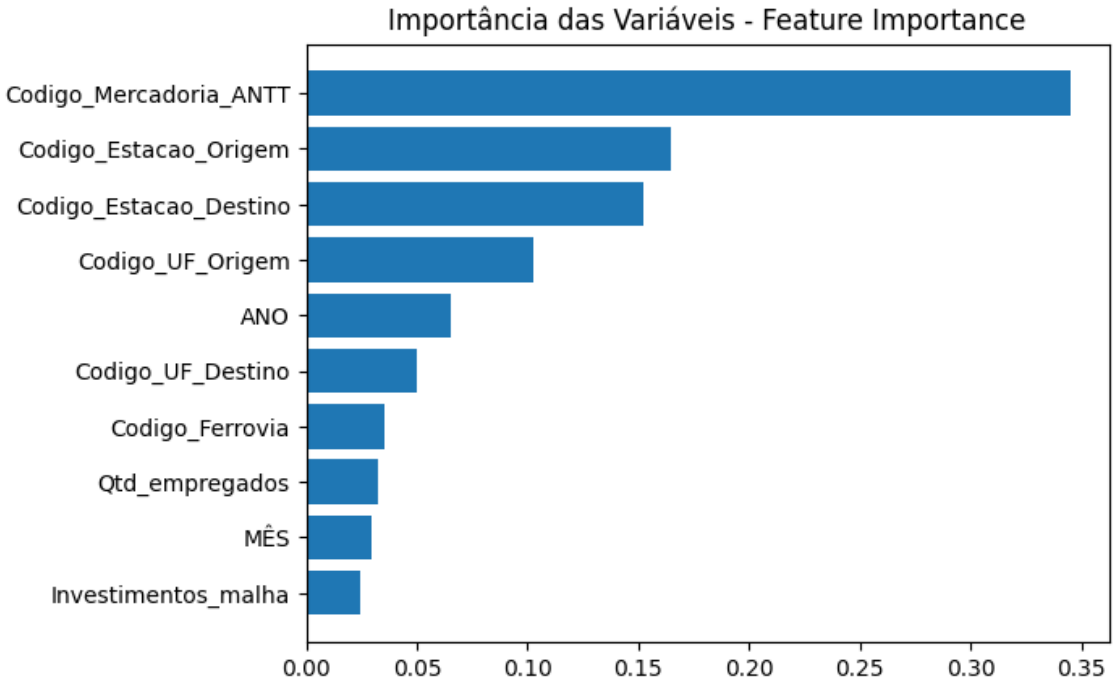
CURVA DE APRENDIZADO mostra a generalização do modelo em relação aos dados de treino x teste. Se o modelo está aprendendo bem ou mal.



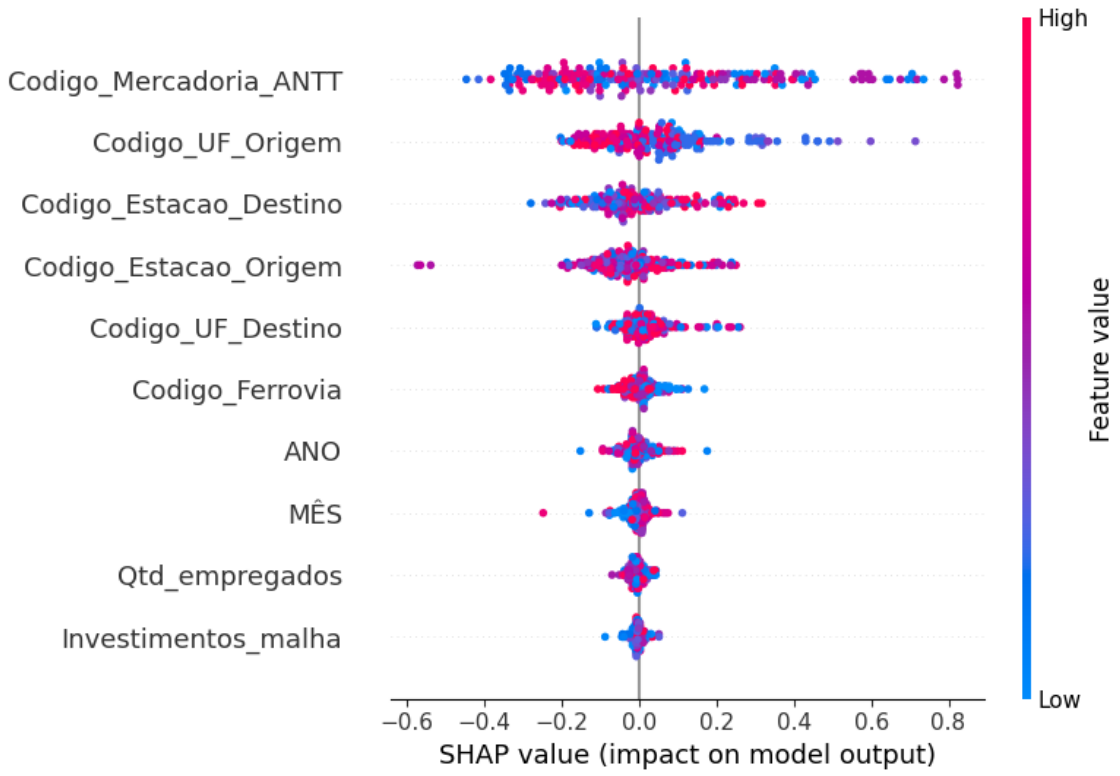
Na imagem podemos observar que o modelo está aprendendo demais o treino, mas não aprende padrões gerais, por isso ele generaliza mal nos registros novos/teste.

Isso caracteriza Overfitting (sobreajuste) pois o modelo aprendeu demais no treino e fixou aquilo como regra.

FEATURE IMPORTANCE permite verificar quais variáveis estão sendo mais benéficas para as predições. Na imagem observamos que o código da mercadoria é a coluna mais relevante para o modelo.



SHAP é uma técnica que verifica a contribuição individual de cada variável para a predição de um modelo. Ele responde à pergunta: "Quanto cada feature influenciou para aumentar ou diminuir o valor previsto?" Na imagem podemos observar que a coluna código de mercadoria influencia muito para uma predição de valores baixos, tendo uma concentração de bolinhas azuis abaixo de zero.

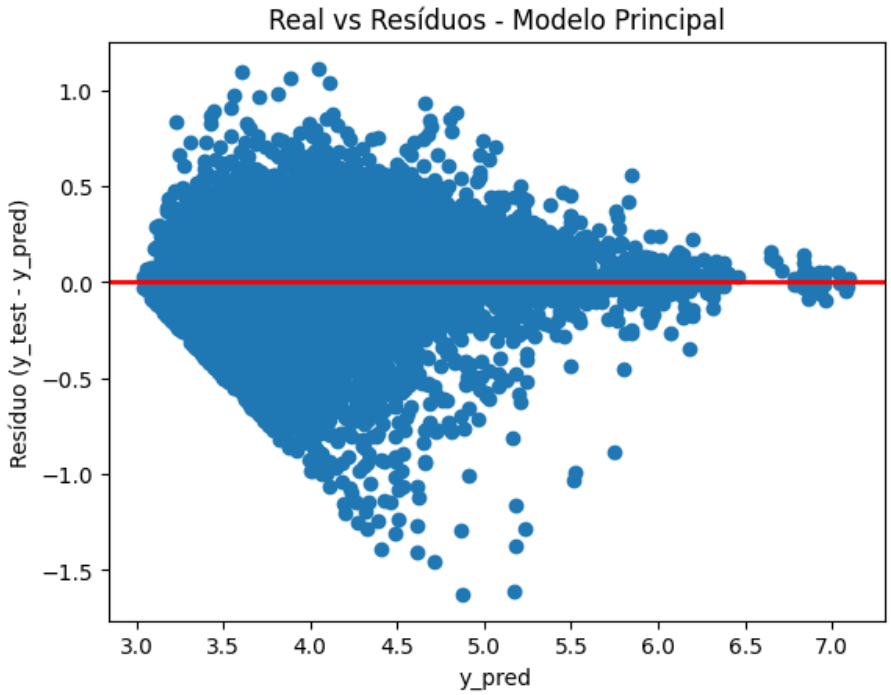


KOLMOGOROV SMIRNOV é um teste para comparar as distribuições do conjunto de treino e teste (apenas as features). Será que ao longo do meu ETL ou com recebimento de novos registros, a distribuição dos meus dados mudou? Se p < 0,05 → Há evidência de mudança significativa (COM drift / Distribuição alterada) ..... se p ≥ 0,05 → Não há evidência de mudança significativa (SEM drift / Distribuição mantida).

O nosso resultado de kolmogorov para as features mostra que a coluna **Codigo_Estacao_Origem** sofreu drift, mas todas as outras mantiveram suas distribuições.

```
################ KOLMOGOROV SMIRNOV ################
MÊS: KS_stat=0.0087, p_value=0.438319
ANO: KS_stat=0.0055, p_value=0.928112
Codigo_Ferrovia: KS_stat=0.0040, p_value=0.996891
Codigo_Mercadoria_ANTT: KS_stat=0.0116, p_value=0.141380
Codigo_Estacao_Origem: KS_stat=0.0143, p_value=0.036560
Codigo_UF_Origem: KS_stat=0.0084, p_value=0.484142
Codigo_Estacao_Destino: KS_stat=0.0089, p_value=0.418075
Codigo_UF_Destino: KS_stat=0.0055, p_value=0.926525
Qtd_empregados: KS_stat=0.0064, p_value=0.819735
Investimentos_malha: KS_stat=0.0058, p_value=0.895942
####################################
```

ANÁLISE DOS RESIDUOS um resíduo é a diferença entre o valor real e o valor que o modelo previu. Na imagem abaixo o gráfico demonstra HETEROCEDASTICIDADE pois os resíduos estão muito espalhados pelo gráfico, parecendo um funil ou tempestade. Isso significa que o modelo erra mais para algumas classes do que em outras, os riscos de usar um modelo que erra forte uma determinada classe é algo delicado a se pensar.



Como os outros resultados do modelo se mostraram favoráveis, uma solução para contornar essa heterocedasticidade seria criar um novo modelo refinado, usando apenas os erros do primeiro modelo, assim ele atuaria corretivamente nas instancias com erros grandes. Porém neste estudo de caso não abordaremos a criação de um novo modelo refinado, pois já utilizamos o RandomizedSearchCV ***(AUTOTUNNING).***

Cenário Fictício - aplicando o modelo

O modelo que foi treinado utilizando informações de 2006 até 2022 tem agora a missão de prever os valores de 2023. Após realizado as transformações necessárias (ETL) é preciso enquadrar o conjunto de dados com a mesma estrutura/ordem de colunas em que o modelo principal foi treinado.

Após aplicado, os resultados do modelo ao cenário de 2023 são observados na imagem abaixo:

```
----------  RESULTADOS  ----------
O tipo do modelo carregado é: <class 'sklearn.ensemble._forest.RandomForestRegressor'>
R² no conjunto de teste: 0.84
MAE: 22111.69819789329
RMSE: 85461.26689426515
MAPE: 58.91%
MedAE: 3356.95
```

O modelo consegue explicar cerca de 84% da variável alvo (TU - toneladas) e isso é melhor do que um chute aleatório, mas conta com um MAPE de 58%, o que significa que quando ele erra o modelo erra 58% a mais do valor real, exemplo o valor real é 100 mas o modelo previu 158. Isso pode indicar que variáveis importantes estão faltando ou que os dados têm muita variabilidade (o nosso caso é variabilidade).

O **ERRO ABSOLUTO MEDIO - (MAE mean absolute error)** retorna que o modelo, em média, erra cerca de 22.111 unidades na previsão de toneladas. Porém esse valor deve ser interpretado em relação à escala dos dados — se TU varia de 0 a milhões, esse erro pode ser tolerável ... se não, pode ser alto.

O **RMSE - Raiz do erro quadrático médio - (Raise mean squared error)** penaliza mais forte os grandes erros, para o nosso modelo obtemos RMSE de 85.461.....Um RMSE muito maior que o MAE indica que há outliers ou erros muito grandes em alguns casos e que estão influenciando performance, porém no nosso caso o RMSE ficou menor do que o MAE então é um risco que podemos seguir dada à variabilidade dos dados.

O **MAPE** é o erro médio (MAE) só que em percentual, para o nosso caso ficou em 58,91%.

Conclusão

O nosso modelo consegue acertar cerca de 84% das previsões, porém para os outros 16%, quando ele erra ele incrementa muitas unidades além do valor real, cerca de 58% do valor real.

Neste sentido, para o cenário proposto o modelo é considerado estável, positivo, pois temos dados muito variados, a escala gigantesca e poucas colunas para explicação. Também porque utilizando o processo de AUTOTUNNING, encontramos parâmetros que melhoraram a performance do modelo e trouxeram maior assertividade e menor erro. A Heterocedasticidade nos resíduos é explicada também por estes pontos.